

A comparative study of feature selection and feature extraction techniques for IoT Intrusion Detection Systems

Monika Vishwakarma^a , Divyansh Dwivedi^b, Nishtha Kesswani^b

^a Department of Computer Applications, Manipal University Jaipur, Jaipur, Rajasthan, India

^b Department of Data Science and Analytics, Central University of Rajasthan, Ajmer, India

ARTICLE INFO

Keywords:

Internet of Things (IoT)
Intrusion Detection System (IDS)
Feature selection
Feature extraction
Machine learning

ABSTRACT

The rapid expansion of Internet of Things (IoT) networks has elevated the threat of cyberattacks, underscoring the necessity for efficient Intrusion Detection Systems (IDS). This study provides a comprehensive analysis that contrasts various feature selection (FS) and feature extraction (FE) methods for IoT intrusion detection. Employing Benchmark IoT IDS datasets, we assess FS techniques including Mutual Information, ANOVA, Chi-Square, Random Forest Importance, and Recursive Feature Elimination (RFE), in addition to FE methods such as PCA, IncrementalPCA, FastICA, LDA, NMF, and TSVD. Experimental findings reveal the trade-offs between FS and FE methods, offering practical insights for choosing the best feature engineering strategies in actual IoT IDS applications.

1. Introduction

In the 21st century, billions of smart devices and sensors connecting through heterogeneous networks represent the most transformative technology, known as the Internet of Things (IoT). These IoT devices provide facilities like automation and data exchange across different domains such as transportation, the medical sector, and smart cities [1]. However, the growth of IoT infrastructure has also increased the chances of cyberattacks, exposing networks to vulnerabilities such as Distributed Denial of Service (DDoS) attacks, data tampering, eavesdropping, and botnet-based intrusions [2].

In traditional security mechanisms, such as rule-based systems, firewalls are often found to be insufficient for detecting novel attacks due to their limited adaptability [3]. Therefore, Intrusion Detection Systems (IDSs) have become popular for security architectures due to their adaptable nature for attack detection. Modern IDS systems increasingly integrate machine learning and deep learning techniques for anomaly detection. However, these system also faces some challenges related to high-dimensional data, imbalanced distribution of benign or malicious data, missing values, computational overhead, and interpretability, all of which directly affect the real-time applicability in IoT scenarios [4].

IoT datasets typically contain a high number of network traffic features that may be redundant or irrelevant. These features can degrade the performance of IDS by increasing model complexity, the risk of

overfitting, and training time. Therefore, feature engineering comes into play to overcome this issue by comprising feature selection and feature extraction of suitable features from the data [5].

The lack of comprehensive evaluation for feature engineering in the existing research on IoT intrusion detection using ML and DL model. Specifically, There is no consistent understanding of which feature selection or extraction techniques yield optimal results across diverse IoT datasets. Similarly, There is no such way to handle missing values in a dataset. Hence, there is a need for a structured, comparative framework to evaluate how different feature selection and feature extraction techniques affect the overall robustness and efficiency of IoT-based IDS models.

The major contributions of this work are as follows:

- We evaluate a diverse set of feature selection and feature extraction techniques under consistent experimental settings for IoT intrusion detection.
- Through systematic removal and substitution of feature engineering components, we quantify their individual impact on IDS performance.
- The study offers evidence-based insights to inform the design of robust and efficient feature engineering pipelines for real-world IoT IDS applications.

* Corresponding author.

E-mail addresses: monika.vishwakarma@jaipur.manipal.edu (M. Vishwakarma), divyanshdiwedi10@gmail.com (D. Dwivedi), nishtha@curaj.ac.in (N. Kesswani).

<https://doi.org/10.1016/j.array.2026.101068>

Received 21 April 2026; Received in revised form 22 June 2026; Accepted 5 July 2026

Available online 10 July 2026

2590-0056/© 2026 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

2. Related work

The expansion of IoT devices across various domains such as smart cities, healthcare, industrial automation, and smart homes has created significant security challenges. IoT networks are vulnerable to cyberattacks due to resource constraints, heterogeneous device architectures, and the massive volume of network traffic generated [6,7]. IDS have emerged as essential security mechanisms for identifying and mitigating malicious activities in IoT environments [8].

Machine learning (ML) and deep learning (DL) approaches have shown significant promise for IoT intrusion detection by enabling the learning of complex patterns from network traffic data and adaptation to evolving attack vectors [9]. However, the high dimensionality of network traffic features imposes computational limitations on IoT devices, necessitating effective feature engineering strategies. Feature selection and feature extraction techniques are the preprocessing steps that reduce dimensionality, eliminate redundant or irrelevant features, improve model accuracy, and decrease computational overhead [10]. This literature review provides a comprehensive analysis of feature selection and feature extraction techniques applied to IoT intrusion detection systems.

2.1. IoT security challenges

The heterogeneous nature of IoT devices, ranging from simple sensors to complex actuators, creates diverse attack surfaces. Resource constraints in terms of processing power, memory, and battery life limit the deployment of computationally intensive security mechanisms [11]. The massive scale of IoT deployments generates enormous volumes of network traffic, making real-time intrusion detection computationally challenging [12].

Common attack types targeting IoT networks include Distributed Denial of Service (DDoS), Denial of Service (DoS), botnet infections (e.g., Mirai), reconnaissance attacks, brute force attacks, Man-in-the-Middle (MITM), injection attacks, ransomware, and protocol-specific exploits [13]. These attacks exploit vulnerabilities in IoT protocols, weak authentication mechanisms, and inadequate security configurations.

2.2. Feature engineering in machine learning

Feature engineering technique contains two primary methods: feature selection and feature extraction. Feature selection recognizes the most relevant subset of original features while discarding redundant or irrelevant features [14]. This approach maintains feature interpretability and reduces computational complexity. Feature selection methods are classified into three categorized: (1) filter methods that use statistical measures independent of learning algorithms (e.g., Information Gain, Chi-Square, Fisher Score, Pearson correlation), (2) wrapper methods that evaluate feature subsets using specific classifiers (e.g., Recursive Feature Elimination), and (3) embedded methods that perform feature selection during model training (e.g., Random Forest feature importance) [15].

Moustafa and Slay introduced the UNSW-NB15 dataset and showed that statistical feature selection significantly improves detection performance while reducing redundancy in intrusion detection models [16]. In their subsequent work on the BoT-IoT dataset, they further emphasized the importance of correlation-based and information-gain feature selection for botnet detection in IoT environments [17]. Shone et al. [18] proposed a non-symmetric deep autoencoder-based feature-extraction method that significantly outperformed traditional machine learning models. More recently, Li et al. [10] conducted direct comparisons between feature selection and feature extraction techniques on TON-IoT datasets, reporting that feature extraction achieved greater robustness while requiring fewer computational resources. Despite significant advances, challenges remain in dataset representatives, class

imbalance, and computational constraints. The findings of this review provide actionable guidance for IoT security. By carefully selecting feature engineering techniques appropriate to deployment contexts, using hybrid filter methods and ensemble classifiers, addressing class imbalance, and validating across multiple datasets, it is possible to develop effective, efficient, and scalable intrusion detection systems that protect the IoT ecosystem from evolving cyber threats.

2.3. Comparative studies of feature selection and feature extraction

Despite the maturity of the individual techniques, comparatively few studies place feature selection and feature extraction under a single, controlled evaluation, and those that do report nuanced, often dataset-dependent conclusions. The work most directly comparable to ours is that of Li et al. [10], who compared FS and FE on the network partition of ToN-IoT for both binary and multi-class detection. They found that feature extraction yields better detection performance than feature selection when the number of retained features is small and is less sensitive to changes in that number, whereas feature selection achieves lower model training and inference time. Vuong et al. [9] reached a related conclusion on UNSW-NB15, and the extended journal study by Ngo et al. [19] confirmed that feature selection attains better detection and lower runtime as the number of retained features grows, while feature extraction is more reliable when only a handful of features are retained.

Broader multi-dataset comparisons reinforce the absence of a universally dominant paradigm. Sarhan et al. [20] evaluated PCA, LDA, and an autoencoder across UNSW-NB15, ToN-IoT, and CSE-CIC-IDS2018 with six models, observing that no single extraction method or classifier was best across all datasets, that accuracy typically stabilized after only three to ten dimensions, and that LDA degraded performance on two of the three datasets; they argued for standardized benchmark feature sets to enable fair comparison. Most recently, Vo et al. [21] benchmarked seventeen feature-selection methods and ten ML models across seven IDS datasets, including ToN-IoT and BoT-IoT, and concluded that tree-based classifiers combined with non-ranking selection methods provide the most consistent performance. Collectively, these studies establish that the relative merit of FS and FE is contingent on the dataset's class structure, dimensionality, and the chosen classifier, which is precisely the motivation for evaluating a broad method set across two structurally different IoT benchmarks under a common pipeline, as we do here.

2.4. Intrusion detection on the ToN-IoT and CIC-IoT-2022 benchmarks

The ToN-IoT dataset was introduced to provide a realistic, heterogeneous testbed spanning telemetry, operating-system logs, and network traffic for AI-based security research [22,23], and Booi et al. [24] subsequently highlighted the heterogeneity of its sources and the consequent need to standardize features and attack types across IoT IDS datasets. Recent ToN-IoT studies increasingly couple the dataset with explicit feature engineering: Almotairi et al. [25] combined ML-based feature selection with stacked ensemble classifiers to improve detection, while Zhang et al. [26] proposed a two-stage selection scheme followed by a sparrow-search optimizer feeding a Bayesian-optimized LightGBM classifier, achieving 97.22% accuracy on ToN-IoT and 96.08% on UNSW-NB15 with a model under 2 MB. These results confirm that ToN-IoT remains a demanding, actively used benchmark for which feature engineering is decisive.

The CIC-IoT-2022 dataset, developed by the Canadian Institute for Cybersecurity, profiles a heterogeneous set of real IoT devices under controlled conditions and was designed primarily for device identification, profiling, and security analysis [27]. Kostas et al. [28] used it to externally validate a device-identification methodology, reporting strong F1-scores and emphasizing that data diversity is a primary determinant of model performance. It is important to distinguish

CIC-IoT-2022 from the later and frequently conflated CIC-IoT-2023 dataset [29], which targets large-scale attacks with 33 attack types organized into seven categories and 46 flow features; the two datasets differ in purpose, device coverage, attack taxonomy, and feature schema, and conclusions drawn on one do not transfer automatically to the other. Notably, attack-detection studies that apply a systematic FS/FE comparison specifically to CIC-IoT-2022 remain scarce relative to the volume of work on CIC-IoT-2023 and ToN-IoT—a gap that the present study directly addresses by evaluating both ToN-IoT and CIC-IoT-2022 under identical conditions.

3. Methodology

In this section, we explain the feature selection and feature extraction techniques employed in the study. The objective is to systematically compare the performance and efficiency of various feature engineering methods applied to IoT intrusion detection systems (IDS). The proposed framework follows a structured pipeline consisting of data preprocessing, application of feature selection (FS) and feature extraction (FE) techniques, model training, and performance evaluation.

3.1. Feature selection (FS) techniques

Feature selection aims to identify the most relevant and discriminative subset of features from the original dataset to improve model interpretability, reduce overfitting, and decrease computational cost. Formally, given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$ where $x_i \in \mathbb{R}^m$ represents an m -dimensional feature vector and y_i denotes the corresponding class label, the objective of feature selection is to determine a subset $S \subseteq \{1, 2, \dots, m\}$ that maximizes a performance criterion, typically related to classification accuracy or mutual dependency between features and class labels.

3.1.1. ANOVA (Analysis of Variance)

ANOVA evaluates whether the means of a particular feature across different classes differ significantly [30,31]. For each feature X_j , the F-statistic is computed as:

$$F_j = \frac{SS_B/(k-1)}{SS_W/(N-k)} \quad (1)$$

where, Between-Class Sum of Squares (SS_B) and Within-Class Sum of Squares (SS_W) calculated as:

$$SS_B = \sum_{i=1}^k n_i(\bar{x}_i - \bar{x})^2,$$

$$SS_W = \sum_{i=1}^k \sum_{x \in G_i} (x - \bar{x}_i)^2,$$

k is the number of classes, N is the total number of samples, G_i is the set of all samples belonging to class i , n_i is the number of samples in class i , $\bar{x}_i$ is the mean of class i , and $\bar{x}$ is the overall mean. A higher F_j indicates stronger class discrimination capability for feature X_j .

3.1.2. Mutual information (MI)

Mutual Information (MI) is a filter-based feature selection technique rooted in information theory that quantifies the statistical dependency between a feature and the target class label. MI detects arbitrary statistical dependencies, making it particularly suitable for non-linear and complex IoT network traffic data [32].

Formally, given a dataset $D = \{(x_i, y_i)\}_{i=1}^n$, the Mutual Information between a feature X_j and the class label Y is defined as:

$$I(X_j; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \quad (2)$$

where $p(x, y)$ is the joint probability distribution of X_j and Y , and $p(x)$ and $p(y)$ are their respective marginal distributions. A value of

$I(X_j; Y) = 0$ indicates complete statistical independence between the feature and the target, while larger values reflect stronger dependency and thus higher discriminative utility.

MI can equivalently be expressed in terms of entropy as:

$$I(X_j; Y) = H(Y) - H(Y | X_j) \quad (3)$$

where $H(Y)$ denotes the Shannon entropy of the class label:

$$H(Y) = - \sum_{y \in \mathcal{Y}} p(y) \log p(y) \quad (4)$$

and $H(Y | X_j)$ is the conditional entropy of Y given X_j :

$$H(Y | X_j) = - \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p(x, y) \log p(y | x) \quad (5)$$

Eq. (3) expresses the reduction in uncertainty about the class label Y that is achieved upon observing the feature X_j . Features that maximally reduce label uncertainty are assigned higher MI scores and are consequently ranked as more informative [33].

3.1.3. Chi-square (χ^2) test

The Chi-squared test measures how much observed frequencies (actual data) deviate from expected frequencies by assuming independence between a feature and the target [34,35].

$$\chi^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad (6)$$

where,

$$E_{ij} = \frac{(\text{sum of row } i) \times (\text{sum of column } j)}{N}$$

O_{ij} is the number of samples that belong to the i th category of the feature and the j th class of the target variable, and E_{ij} is the expected frequency. A higher χ^2 value signifies a stronger association between the feature and the target variable.

3.1.4. Random Forest importance

Random Forest measures feature importance by quantifying each feature's contribution to impurity reduction across all trees in the ensemble [36,37]. The feature importance is calculated as:

$$I(X_j) = \frac{1}{T} \sum_{t=1}^T \sum_{n \in N_t(X_j)} \frac{N_n}{N} \Delta i(n) \quad (7)$$

where T is the total number of trees, $N_t(X_j)$ is the set of nodes where feature X_j is used for splitting, N_n is the number of samples at node n , and $\Delta i(n)$ is the impurity decrease (e.g., based on Gini or entropy). Larger $I(X_j)$ values imply greater predictive importance.

3.1.5. Recursive feature elimination (RFE)

RFE is a wrapper-based method that recursively eliminates the least important features based on model coefficients [38]. Given model weights $\mathbf{w} = [w_1, w_2, \dots, w_d]$, the ranking criterion is defined as:

$$R_j = w_j^2 \quad (8)$$

The feature(s) with the smallest R_j are removed iteratively until the desired subset size k is achieved. RFE thus balances model performance with feature interpretability.

3.2. Feature extraction (FE) techniques

Feature extraction transforms the original feature space into a lower-dimensional subspace while retaining discriminative or statistical information. The following FE methods were analyzed.

3.2.1. Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is employed to reduce the dimensionality of the feature space while preserving the maximum variance of the data [39]. Let

$$X \in \mathbb{R}^{n \times d}$$

denote the input feature matrix, where n represents the number of samples and d denotes the number of original features.

Prior to dimensionality reduction, the data are mean-centered by subtracting the feature-wise mean vector:

$$\mu = \frac{1}{n} \sum_{i=1}^n X_i, \quad X_c = X - \mu. \quad (9)$$

In this work, PCA is implemented using the scikit-learn module [40], which internally applies Singular Value Decomposition (SVD) to the centered data matrix:

$$X_c = USV^T, \quad (10)$$

where U and V are orthogonal matrices and S is a diagonal matrix containing the singular values.

The principal components correspond to the right singular vectors associated with the largest singular values. The top k components are selected to form the projection matrix:

$$V_k = [v_1, v_2, \dots, v_k]. \quad (11)$$

The variance explained by the i th principal component is computed as:

$$\lambda_i = \frac{s_i^2}{n-1}, \quad (12)$$

which is consistent with the variance estimation used in scikit-learn [40].

The reduced-dimensional representation of the data is obtained by projecting the centered data onto the selected components:

$$Z = X_c V_k \quad (13)$$

In the experimental implementation, PCA is applied with $k = 10$ principal components, providing a compact and informative feature representation for subsequent learning tasks.

3.2.2. Incremental Principal Component Analysis (IPCA)

Incremental Principal Component Analysis (IPCA), implemented, is a memory-efficient extension of Principal Component Analysis (PCA) designed for large-scale and streaming datasets. Unlike standard PCA, which requires loading the entire dataset into memory, IPCA processes data in mini-batches and incrementally updates the principal components [40].

Given a dataset $X \in \mathbb{R}^{n \times d}$, classical PCA computes principal components via Singular Value Decomposition (SVD):

$$X = USV^T \quad (14)$$

where the top k columns of V correspond to the principal directions.

In IPCA, the dataset is divided into batches X_t of size b . A running mean is updated incrementally as:

$$\mu_t = \frac{n_{t-1}\mu_{t-1} + b\bar{x}_t}{n_t} \quad (15)$$

For each batch, an augmented matrix is formed by combining previous components with newly centered data:

$$B = \begin{bmatrix} \Sigma_{t-1} V_{t-1}^T \\ X_t - \mu_t \end{bmatrix} \quad (16)$$

SVD is then applied:

$$B = \tilde{U} \tilde{\Sigma} \tilde{V}^T \quad (17)$$

and the top k components of $\tilde{V}$ are retained as updated principal components.

3.2.3. Independent Component Analysis (ICA)

Independent Component Analysis (ICA) aims to decompose multivariate observations into statistically independent components by maximizing non-Gaussianity [41]. Let

$$X \in \mathbb{R}^{n \times d}$$

denote the observed data matrix generated as a linear mixture of latent independent sources:

$$X = AS, \quad (18)$$

where A is an unknown mixing matrix and S contains statistically independent components.

ICA estimates an unmixing matrix $W = A^{-1}$ such that:

$$S = WX. \quad (19)$$

Prior to optimization, the data are centered and whitened to remove second-order correlations. The independent components are then estimated by maximizing non-Gaussianity, commonly measured using an approximation of negentropy. For each component w_i , the objective function is defined as:

$$J(w_i) = \left| \mathbb{E}\{G(w_i^T X_w)\} - \mathbb{E}\{G(v)\} \right|, \quad (20)$$

where X_w denotes the whitened data, v is a standard Gaussian random variable, and $G(\cdot)$ is a suitable nonlinear function.

FastICA employs a fixed-point iteration scheme to maximize this objective efficiently, yielding statistically independent latent components.

3.2.4. Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) seeks a linear projection that maximizes between-class variance while minimizing within-class variance [42]. The optimal projection matrix W is obtained by maximizing the Fisher criterion:

$$W^* = \arg \max_W \frac{|W^T S_B W|}{|W^T S_W W|}, \quad (21)$$

where S_B and S_W denote the between-class and within-class scatter matrices, respectively.

The between-class scatter matrix is defined as:

$$S_B = \sum_{i=1}^k n_i (\mu_i - \mu)(\mu_i - \mu)^T, \quad (22)$$

and the within-class scatter matrix is given by:

$$S_W = \sum_{i=1}^k \sum_{x \in C_i} (x - \mu_i)(x - \mu_i)^T, \quad (23)$$

where k is the number of classes, n_i denotes the number of samples in class C_i , μ_i is the mean of class C_i , and μ is the global mean.

The discriminant directions are obtained by solving the generalized eigenvalue problem:

$$S_B w = \lambda S_W w. \quad (24)$$

The number of extracted components is limited to at most $k-1$, and the projection matrix is learned using labeled training data and subsequently applied to the test data.

3.2.5. Non-negative Matrix Factorization (NMF)

Non-negative Matrix Factorization (NMF) decomposes a non-negative data matrix $X \in \mathbb{R}_+^{m \times n}$ into two lower-rank non-negative matrices [43]:

$$X \approx WH, \quad (25)$$

where $W \in \mathbb{R}_+^{m \times r}$ and $H \in \mathbb{R}_+^{r \times n}$, with $r \ll \min(m, n)$. The matrix W represents the extracted latent features, while H contains the corresponding coefficients.

NMF is formulated as an optimization problem that minimizes the reconstruction error under non-negativity constraints:

$$\min_{W, H \geq 0} \|X - WH\|_F^2. \quad (26)$$

In this work, NMF is employed as an unsupervised feature extraction technique, yielding a compact and interpretable representation of the original feature space.

3.2.6. Truncated Singular Value Decomposition (TSVD)

Truncated Singular Value Decomposition (TSVD) [44] approximates a data matrix $X \in \mathbb{R}^{m \times n}$ using only the top k singular values and corresponding singular vectors obtained from its singular value decomposition:

$$X = U \Sigma V^T. \quad (27)$$

By retaining the leading k singular components, TSVD yields a rank- k approximation:

$$X_k = U_k \Sigma_k V_k^T, \quad (28)$$

where $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_k)$ with $\sigma_1 \geq \dots \geq \sigma_k$.

For feature extraction, the reduced-dimensional representation is obtained as:

$$Z = U_k \Sigma_k, \quad (29)$$

which captures the dominant variance structure of the data.

Unlike PCA, TSVD does not require mean-centering and is therefore well suited for large-scale and sparse high-dimensional datasets, such as IoT network traffic.

4. Datasets and preprocessing

We have used two datasets, ToN-IoT and CIC-IoT-2022, for performing a comparative analysis of feature selection and extraction.

4.1. Benchmark IoT IDS datasets

4.1.1. ToN-IoT dataset

The ToN-IoT dataset is a realistic, heterogeneous cybersecurity dataset designed to support AI-based security research in IoT and IIoT environments. The dataset was generated using a distributed testbed that integrates edge, fog, and cloud layers through Software-Defined Networking (SDN), Network Function Virtualisation (NFV), and Service Orchestration [23]. The dataset includes synchronized data from multiple sources, including network traffic, IoT/IIoT sensor telemetry, and audit logs from Windows and Linux operating systems, reflecting the behavior of modern smart-city and Industry 4.0 networks.

ToN-IoT contains both benign activities and nine contemporary cyberattack categories, including scanning, DoS/ DDoS, ransomware, backdoor, injection, cross-site scripting, password attacks, and man-in-the-middle attacks. For the network component, rich flow-level features are extracted using Zeek, application-layer behaviors, and protocol anomalies, making the dataset suitable for evaluating machine learning-based intrusion detection and cybersecurity solutions.

4.1.2. CIC-IoT-2022 dataset

The CIC-IoT-2022 Dataset is a realistic and multidimensional Internet of Things (IoT) dataset developed by the Canadian Institute for Cybersecurity. The dataset was designed to support IoT device identification, profiling, and security analysis by capturing comprehensive network behavior from a heterogeneous set of real-world IoT devices under controlled, realistic conditions [27].

The networking dataset collected from 60 IoT devices representing a wide range of categories, including cameras, smart speakers, smart hubs, sensors, smart plugs, lighting devices, appliances, and other home automation components. The devices operate using multiple communication technologies, namely Wi-Fi, Zigbee, and Z-Wave, allowing the dataset to reflect the protocol diversity commonly found in modern smart environments.

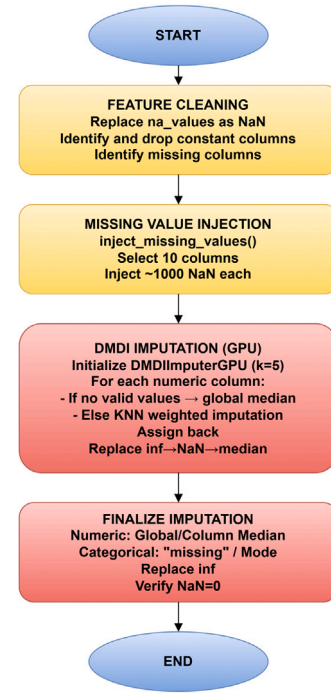


Fig. 1. Preprocessing.

4.2. Data preprocessing

The preprocessing pipeline is essential for preparing the dataset for model training. Our preprocessing methodology consists of several systematic steps designed to handle the inherent challenges of network traffic data, including missing values, categorical variables, and feature scaling. We carefully handle placeholder values commonly found in network traffic data. Values such as “,” “ ”, “F”, “f”, “NA”, “null”, and “None” were explicitly mapped to NaN during loading to ensure consistent missing-value representation across all features (see Fig. 1).

To prevent data leakage and ensure unbiased evaluation, the dataset was stratified with a 70–30 train-test split before any preprocessing transformations. The stratification ensured that the class distribution was preserved in both training and test sets. Features with zero variance, which provide no discriminative information for classification, are identified and removed. This step reduced dimensionality while eliminating redundant features. To evaluate the robustness of the imputation strategy, artificial missing values were injected into a subset of features in the training data by randomly replacing selected values with NaN. Missing value handling was performed using a novel GPU-accelerated Distance-based Mutual Dependency Imputation (DMDI) algorithm. The DMDI approach uses the correlation structure among features to impute missing values more accurately than traditional methods. For each missing value, the algorithm:

1. Identifies the k nearest neighbors based on Euclidean distance in the feature space.
2. Computes inverse distance weights:

$$w = \frac{1}{d + \epsilon}, \quad \text{where } \epsilon = 10^{-9}$$

3. Calculates the imputed value as a weighted average:

$$\hat{x} = \frac{\sum_{j=1}^k w_j y_j}{\sum_{j=1}^k w_j}$$

For columns with no valid values, the global median was used as a fallback. Categorical columns were imputed using the mode. This hybrid approach ensures robustness while preserving the underlying data

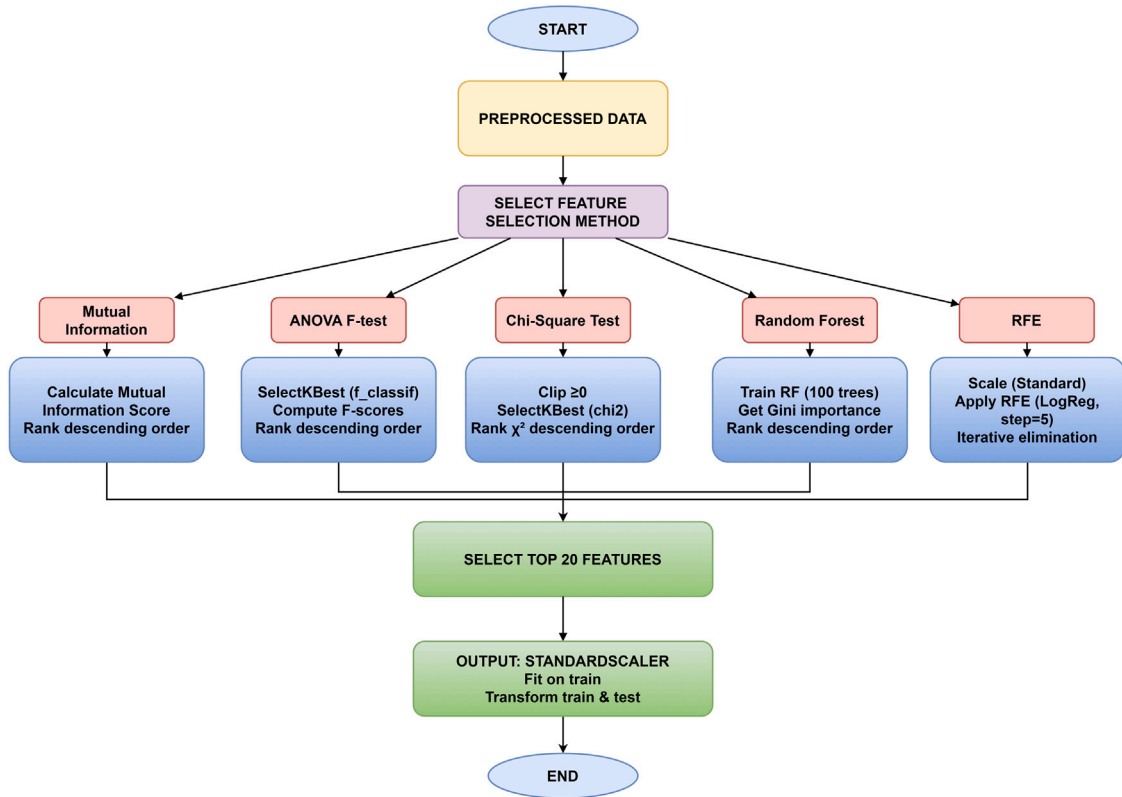


Fig. 2. Feature selection techniques.

distribution. Similarly, all categorical features were encoded to enable compatibility with numerical machine learning algorithms. Standard-Scaler normalization was applied to transform features to have zero mean and unit variance:

$$z = \frac{x - \mu}{\sigma}$$

where x is the original value, μ is the feature mean, and σ is the standard deviation.

4.3. Classification models

We employed six classification models to assess the performance of various dimensionality reduction techniques. These models were chosen to cover a broad range of learning paradigms, including probabilistic approaches, tree-based methods, ensemble learners, and neural networks. Among them, Decision Tree is a non-parametric supervised learning algorithm that creates a tree-like model of decisions based on feature values. It recursively partitions the feature space using the most discriminative features at each node. The algorithm uses Gini impurity as the splitting criterion and was configured with `random_state=42` for reproducibility. Decision Trees are interpretable and handle non-linear relationships well, but are prone to overfitting. Random Forest is an ensemble learning method that constructs multiple decision trees during training and outputs the mode of their predictions. Our implementation used 100 estimators. Random Forest provides robustness against overfitting through bootstrap aggregating and random feature selection, making it particularly suitable for high-dimensional intrusion detection data.

K-Nearest Neighbors (KNN) classifies samples based on the majority class of their five nearest neighbors using Euclidean distance. Since KNN is sensitive to feature scaling, it helps evaluate the impact of preprocessing and dimensionality reduction. Gaussian Naive Bayes (GNB) is a simple probabilistic classifier based on Bayes' theorem with assumptions of feature independence and Gaussian distributions.

It serves as a fast and interpretable baseline for comparison. The MLP classifier is a neural network trained using backpropagation. We used the default architecture with `max_iter = 300` to allow the model to converge. MLP can learn complex non-linear patterns in the data through its hidden layers, but requires careful hyperparameter tuning and sufficient training data.

5. Results and analysis

In this section we discussed the feature selection and feature extraction results with their comparative analysis.

5.1. Feature selection results

For feature selection analysis, we have used five feature selection techniques namely Mutual Information (MI), Anova F-test, Chi-Square, Random Forest feature importance, and Recursive Feature Elimination (RFE) methods as shown in Fig. 2. Each method selects top twenty features from the datasets. These twenty selected features are used for model training and testing to perform the comparative analysis.

Table 1 presents the attack-wise performance using Mutual Information features on the ToN-IoT dataset. Random Forest achieved the best overall performance, with F1-scores above 0.85 for most attack classes and 0.94 for normal traffic. Decision Tree also performed well but showed weaker detection for Ransomware (F1 = 0.23) and MITM (F1 = 0.52). KNN achieved good results for DDoS and Normal traffic but struggled with several attacks, yielding low F1-scores for Backdoor, Injection, and Scanning. Gaussian Naive Bayes performed poorly across most classes, while MLP produced moderate results with strong performance on DDoS and Normal traffic but weak detection for several attack categories.

Table 2 presents the attack-wise performance using Chi-Square feature selection on the ToN-IoT dataset. Decision Tree, Random Forest, and KNN achieved excellent results with F1-scores above 0.9 for most

Table 1

Attack-wise performance using Mutual Information Features on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	1.00	1.00	0.98	1.00	0.99	0.89	0.01	0.02	0.13	0.99	0.23	0.82	0.01	0.01
DDoS	0.91	0.92	0.91	0.97	0.96	0.96	0.94	0.76	0.84	0.60	0.24	0.34	0.82	0.90	0.86
DoS	0.67	0.94	0.78	0.78	0.98	0.87	0.95	0.19	0.31	0.00	0.00	0.00	0.42	0.88	0.57
Injection	0.88	0.88	0.88	0.98	0.95	0.96	0.24	0.12	0.16	0.32	0.86	0.46	0.03	0.02	0.03
MITM	0.57	0.48	0.52	0.70	0.74	0.72	0.38	0.42	0.40	0.02	0.72	0.03	0.39	0.43	0.41
Normal	0.96	0.66	0.78	0.89	0.99	0.94	0.84	0.97	0.90	1.00	0.12	0.22	0.88	0.93	0.91
Password	0.97	0.97	0.97	1.00	0.74	0.85	0.69	0.21	0.32	0.16	0.00	0.00	0.61	0.21	0.32
Ransomware	0.14	0.70	0.23	0.89	0.44	0.58	0.08	0.09	0.08	0.00	0.00	0.00	0.07	0.00	0.01
Scanning	0.98	0.95	0.96	0.92	0.19	0.32	0.10	0.22	0.14	0.00	0.00	0.00	0.22	0.02	0.03
XSS	0.62	0.88	0.73	0.81	0.39	0.53	0.70	0.33	0.44	0.11	0.89	0.19	0.36	0.97	0.52

Table 2

Attack-wise performance using Chi-Square Feature Selection on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.01	0.01	0.01	0.48	1.00	0.64
DDoS	0.98	0.97	0.97	0.99	0.97	0.98	0.97	0.97	0.97	0.39	0.24	0.30	0.94	0.93	0.94
DoS	0.98	0.98	0.98	0.98	0.98	0.98	0.99	0.98	0.98	0.01	0.03	0.02	0.78	0.24	0.36
Injection	0.96	0.96	0.96	0.97	0.96	0.97	0.97	0.96	0.96	0.00	0.00	0.00	0.92	0.88	0.90
MITM	0.67	0.63	0.65	0.72	0.73	0.73	0.64	0.63	0.63	0.01	0.69	0.02	0.53	0.46	0.50
Normal	0.99	0.99	0.99	0.99	0.99	0.99	1.00	0.99	0.99	1.00	0.04	0.07	0.98	0.94	0.96
Password	0.98	0.98	0.98	0.99	0.98	0.98	0.98	0.97	0.98	0.27	0.66	0.38	0.89	0.62	0.73
Ransomware	0.85	0.93	0.89	0.85	0.93	0.89	0.80	0.97	0.88	0.08	1.00	0.15	0.78	0.88	0.83
Scanning	0.98	0.99	0.98	0.98	0.99	0.99	0.99	0.99	0.99	0.00	0.00	0.00	0.70	0.79	0.74
XSS	0.94	0.90	0.92	0.94	0.91	0.92	0.93	0.90	0.91	0.16	0.05	0.07	0.67	0.90	0.77

Table 3

Attack-wise performance using ANOVA Feature Selection on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	0.27	0.01	0.01	1.00	0.00	0.00	1.00	0.01	0.01	0.13	0.99	0.23	0.44	1.00	0.61
DDoS	0.82	0.92	0.87	0.70	0.94	0.80	0.63	0.27	0.38	0.58	0.24	0.34	0.82	0.90	0.86
DoS	0.14	0.06	0.08	0.39	0.93	0.55	0.80	0.08	0.15	0.00	0.00	0.00	0.42	0.06	0.11
Injection	0.84	0.70	0.76	0.98	0.62	0.76	0.29	0.82	0.43	0.00	0.00	0.00	0.29	0.00	0.01
MITM	0.47	0.49	0.48	0.43	0.68	0.53	0.49	0.35	0.41	0.00	0.97	0.01	0.25	0.54	0.34
Normal	0.94	0.94	0.94	0.87	0.97	0.91	0.86	0.90	0.88	1.00	0.09	0.16	0.93	0.85	0.89
Password	0.90	0.68	0.77	1.00	0.46	0.63	0.93	0.05	0.10	0.14	0.00	0.00	0.27	0.74	0.39
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.01	0.03	0.00	0.00	0.00	0.12	0.15	0.13
Scanning	0.10	0.23	0.13	0.59	0.02	0.05	0.10	0.22	0.14	0.00	0.00	0.00	0.53	0.01	0.02
XSS	0.37	0.95	0.53	0.64	0.86	0.73	0.18	0.26	0.21	0.27	0.01	0.01	0.49	0.80	0.61

attack classes, with Random Forest providing the best overall performance. Gaussian Naïve Bayes performed poorly, yielding low F1-scores for most classes despite moderate detection of Password (F1 = 0.38) and Ransomware (F1 = 0.15). MLP showed competitive performance with high F1-scores for DDoS (0.94), Injection (0.90), Normal traffic (0.96), and Ransomware (0.83), but weaker results for DoS (0.36) and MITM (0.50). Overall, Chi-Square features were highly effective for tree-based models and KNN, while Gaussian NB remained the least reliable classifier.

Table 3 presents the attack-wise performance using ANOVA feature selection on the ToN-IoT dataset. Random Forest achieved the best overall performance, with strong detection of DDoS, DoS, Injection, Normal traffic, and XSS (F1-scores of 0.73–0.91). Decision Tree also performed well, particularly for DDoS, Injection, Normal traffic, Password, and XSS. KNN showed moderate performance and struggled with several attack classes, while Gaussian Naïve Bayes performed poorly across most categories. MLP achieved reasonable results for Backdoor, DDoS, Normal traffic, and XSS but remained ineffective for several attack types. Overall, Random Forest emerged as the most effective classifier with ANOVA-selected features.

Table 4 presents the attack-wise performance using Random Forest feature importance on the ToN-IoT dataset. Decision Tree achieved strong performance for most classes, with F1-scores ranging from 0.79 to 1.00, though performance was weak for MITM (0.06), Scanning

(0.24), and XSS (0.32). KNN showed moderate results, performing best on Normal traffic (F1 = 0.90) but yielding low F1-scores for several attacks. Gaussian Naïve Bayes remained ineffective, with most F1-scores below 0.50. MLP provided better overall performance than KNN and NB, achieving high F1-scores for DDoS (0.91) and Normal traffic (0.92), but struggled on Injection, Scanning, and XSS. Overall, Decision Tree and MLP performed comparatively better, whereas KNN and Gaussian NB showed limited effectiveness with RF-selected features.

Table 5 presents the attack-wise performance using Recursive Feature Elimination (RFE) on the ToN-IoT dataset. Random Forest achieved the best overall performance, with F1-scores above 0.95 for most attack classes, while Decision Tree also performed strongly but showed lower performance for Injection (0.63), Ransomware (0.74), and XSS (0.34). KNN provided moderate results, performing well for Normal traffic (F1 = 0.90) and DoS (F1 = 0.92) but struggling with Injection, Scanning, and XSS. Gaussian Naïve Bayes remained ineffective, with most F1-scores close to zero. MLP achieved good performance for DDoS (0.89), Normal traffic (0.91), Scanning (0.74), and XSS (0.72), but showed inconsistent detection for several attack classes. Overall, Random Forest was the most reliable model with RFE-selected features.

Table 6 presents the attack-wise performance using Mutual Information features on the CIC-IoT-2022 dataset. Decision Tree and Random Forest achieved excellent results, with F1-scores of 0.85–1.00

Table 4

Attack-wise performance using Random Forest Feature Importance on ToN-IoT dataset.

Attack type	Decision tree			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	1.00	1.00	0.89	0.01	0.02	0.13	0.99	0.23	1.00	0.00	0.00
DDoS	0.94	0.92	0.93	0.94	0.76	0.84	0.53	0.24	0.33	0.91	0.90	0.91
DoS	0.91	0.98	0.94	0.95	0.19	0.31	0.00	0.00	0.00	0.40	0.94	0.56
Injection	0.71	0.88	0.79	0.24	0.13	0.16	0.30	0.87	0.45	0.90	0.01	0.02
MITM	0.03	0.44	0.06	0.38	0.43	0.40	0.02	0.75	0.05	0.48	0.43	0.45
Normal	0.96	0.62	0.75	0.84	0.97	0.90	1.00	0.16	0.28	0.92	0.92	0.92
Password	0.89	0.97	0.93	0.69	0.21	0.32	0.16	0.00	0.00	0.39	0.73	0.51
Ransomware	0.14	0.79	0.24	0.08	0.09	0.08	0.00	0.00	0.00	0.96	0.06	0.12
Scanning	0.98	0.95	0.96	0.10	0.22	0.14	0.00	0.00	0.00	0.78	0.06	0.11
XSS	0.36	0.29	0.32	0.70	0.33	0.44	0.11	0.88	0.19	0.37	0.93	0.53

Table 5

Attack-wise performance using Recursive Feature Elimination on ToN IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	0.99	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.13	1.00	0.23	1.00	0.01	0.01
DDoS	0.87	0.91	0.89	0.96	0.96	0.96	0.95	0.56	0.70	0.78	0.24	0.36	0.89	0.89	0.89
DoS	0.95	0.98	0.96	0.99	0.98	0.98	0.91	0.93	0.92	0.00	0.00	0.00	0.38	0.92	0.54
Injection	0.88	0.49	0.63	0.97	0.78	0.86	0.70	0.34	0.45	0.22	0.95	0.35	0.45	0.05	0.10
MITM	0.49	0.48	0.49	0.67	0.57	0.62	0.60	0.52	0.56	0.01	0.92	0.01	0.64	0.16	0.25
Normal	0.93	0.99	0.96	0.92	0.99	0.96	0.82	0.99	0.90	1.00	0.09	0.17	0.88	0.94	0.91
Password	0.90	0.94	0.92	0.99	0.96	0.98	0.91	0.68	0.78	0.00	0.00	0.00	0.69	0.44	0.54
Ransomware	0.89	0.63	0.74	0.89	0.45	0.59	0.89	0.45	0.60	0.00	0.00	0.00	0.75	0.16	0.26
Scanning	0.98	0.99	0.98	0.98	0.99	0.98	0.88	0.15	0.25	0.00	0.00	0.00	0.70	0.79	0.74
XSS	0.41	0.30	0.34	0.82	0.40	0.54	0.78	0.39	0.52	0.11	0.18	0.14	0.62	0.85	0.72

Table 6

Attack-wise performance using Mutual Information Features on CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.90	0.85	0.78	0.94	0.85	0.12	0.13	0.12	0.74	0.99	0.85
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.79	0.96	0.87	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.78	0.88	1.00	1.00	1.00
TCPFlood	0.88	0.82	0.85	0.88	0.82	0.85	0.92	0.80	0.85	0.58	0.69	0.63	0.97	0.71	0.82
UDPFlood	0.99	0.99	0.99	1.00	0.99	1.00	0.99	0.99	0.99	0.36	0.99	0.52	0.99	0.97	0.98

across all attack categories, while KNN exhibited comparable performance with F1-scores ranging from 0.85 to 1.00. MLP also showed strong overall performance, attaining F1-scores between 0.82 and 1.00, though its performance on TCPFlood was slightly lower. In contrast, Gaussian Naïve Bayes was the weakest model, producing poor results for BruteForce (F1 = 0.12) and moderate performance for TCPFlood and UDPFlood. Overall, tree-based models remained the most effective classifiers, with KNN and MLP providing competitive results on the CIC-IoT-2022 dataset.

Table 7 presents the attack-wise performance using Chi-Square feature selection on the CIC-IoT-2022 dataset. Decision Tree and Random Forest achieved the best overall performance, with F1-scores ranging from 0.75 to 0.99, while KNN showed comparable results with F1-scores between 0.72 and 0.98. Gaussian Naïve Bayes exhibited poor performance for most attack categories, despite high recall for BruteForce and UDPFlood. MLP performed inconsistently, achieving a high F1-score only for Normal traffic (0.68) and very low scores for the remaining classes, including zero F1-score for HTTPFlood and TCPFlood. Overall, tree-based models remained the most effective classifiers, whereas Gaussian Naïve Bayes and MLP were considerably less reliable when using Chi-Square features.

Table 8 presents the attack-wise performance using ANOVA features on the CIC-IoT-2022 dataset. Decision Tree and Random Forest achieved the best overall performance, obtaining F1-scores between 0.66 and 1.00, with excellent results for HTTPFlood, Normal, and UDPFlood. KNN showed competitive performance for HTTPFlood and Normal traffic but exhibited reduced effectiveness for BruteForce and UDPFlood. Gaussian Naïve Bayes performed inconsistently, yielding

moderate results only for TCPFlood and UDPFlood while failing to detect BruteForce traffic. MLP achieved good performance for Normal traffic (F1 = 0.81) and moderate results for HTTPFlood and BruteForce, but completely failed to classify TCPFlood. Overall, tree-based models remained the most effective classifiers with ANOVA features, whereas KNN, Gaussian Naïve Bayes, and MLP exhibited varying levels of degradation across attack categories.

Table 9 presents the attack-wise performance using Random Forest feature importance on the CIC-IoT-2022 dataset. Decision Tree and KNN achieved excellent performance, obtaining F1-scores between 0.85 and 1.00 across all attack categories, with perfect classification of Normal traffic. MLP also showed strong overall performance, with F1-scores ranging from 0.79 to 1.00, although its effectiveness on TCPFlood was slightly lower. In contrast, Gaussian Naïve Bayes remained the weakest classifier, producing low F1-scores for BruteForce (0.12), TCPFlood (0.63), and UDPFlood (0.52), despite relatively good performance for HTTPFlood and Normal traffic. Overall, Random Forest feature importance enabled highly accurate classification for Decision Tree, KNN, and MLP, while Gaussian Naïve Bayes exhibited comparatively inferior performance.

Table 10 presents the attack-wise performance using Recursive Feature Elimination (RFE) on the CIC-IoT-2022 dataset. Decision Tree and Random Forest achieved excellent performance, with F1-scores ranging from 0.85 to 1.00 across all attack categories. KNN also produced comparable results, obtaining F1-scores between 0.85 and 1.00, with perfect classification of Normal traffic. MLP demonstrated strong overall performance, achieving F1-scores from 0.79 to 1.00, although its performance on TCPFlood was relatively lower. In contrast, Gaussian

Table 7

Attack-wise performance using Chi-Square Features CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.69	0.86	0.77	0.70	0.80	0.75	0.63	0.83	0.72	0.06	0.95	0.11	0.78	0.01	0.01
HTTPFlood	0.97	0.96	0.96	0.97	0.97	0.97	0.96	0.96	0.96	0.87	0.28	0.43	0.00	0.00	0.00
Normal	0.99	0.99	0.99	0.99	0.99	0.99	0.98	0.98	0.98	0.91	0.24	0.38	0.51	0.99	0.68
TCPFlood	0.84	0.77	0.80	0.84	0.77	0.81	0.90	0.73	0.81	0.07	0.08	0.07	0.01	0.00	0.00
UDPFlood	0.97	0.97	0.97	0.98	0.97	0.98	0.96	0.91	0.94	0.09	0.84	0.16	0.64	0.03	0.05

Table 8

Attack-wise performance using ANOVA Features on CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.55	0.85	0.66	0.55	0.85	0.66	0.36	0.24	0.29	0.00	0.00	0.00	0.50	0.71	0.59
HTTPFlood	0.95	0.95	0.95	0.95	0.95	0.95	0.86	0.98	0.91	0.45	0.95	0.61	0.98	0.50	0.66
Normal	0.98	0.97	0.97	0.98	0.97	0.97	0.99	0.94	0.96	0.82	0.21	0.33	0.68	1.00	0.81
TCPFlood	0.99	0.70	0.82	0.99	0.70	0.82	0.95	0.66	0.78	0.69	0.66	0.67	0.27	0.00	0.00
UDPFlood	1.00	0.99	0.99	1.00	1.00	1.00	0.87	0.51	0.64	1.00	0.55	0.71	0.98	0.14	0.24

Table 9

Attack-wise performance using Random Forest Feature Importance CIC-IoT-2022.

Class	Decision tree			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.78	0.94	0.85	0.12	0.13	0.12	0.76	0.86	0.81
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.79	0.96	0.87	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.78	0.88	1.00	1.00	1.00
TCPFlood	0.88	0.82	0.85	0.92	0.80	0.85	0.58	0.69	0.63	0.88	0.72	0.79
UDPFlood	0.99	0.99	0.99	0.99	0.99	0.99	0.36	0.99	0.52	0.99	0.99	0.99

Table 10

Attack-wise performance using Recursive Feature Elimination CIC-IoT-2022.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.89	0.85	0.78	0.94	0.85	0.12	0.13	0.12	0.76	0.86	0.81
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.79	0.96	0.87	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.78	0.88	1.00	1.00	1.00
TCPFlood	0.88	0.82	0.85	0.88	0.82	0.85	0.92	0.80	0.85	0.58	0.69	0.63	0.88	0.72	0.79
UDPFlood	0.99	0.99	0.99	1.00	1.00	1.00	0.99	0.99	0.99	0.36	0.99	0.52	0.99	0.99	0.99

Naïve Bayes remained the weakest classifier, showing poor detection of BruteForce and UDPFlood traffic despite moderate performance for HTTPFlood and Normal traffic. Overall, RFE-selected features enabled highly accurate classification for tree-based models, KNN, and MLP, while Gaussian Naïve Bayes exhibited comparatively inferior performance.

5.2. Feature extraction results

For feature extraction analysis, we have used six feature extraction techniques namely PCA, FastICA, Incremental PCA, LDA, NMF, and TSVD methods as shown in Fig. 3. Each method extract top twenty features from the datasets except LDA. LDA choses features equal to number of classes - 1 present in the dataset. These twenty extracted features are used for model training and testing to perform the comparative analysis.

Table 11 presents the attack-wise performance using PCA features on the ToN-IoT dataset. Overall, PCA led to poor class-wise discrimination across all classifiers. Random Forest achieved the best performance among the evaluated models, obtaining moderate F1-scores for DoS (0.53), Normal traffic (0.77), and XSS (0.27), while failing to detect several attack classes. KNN and MLP showed comparable performance, achieving F1-scores of 0.50–0.53 for DoS and moderate detection of XSS, but exhibiting near-zero performance for several attack categories. Decision Tree performed poorly, with acceptable results only for Normal traffic (F1 = 0.73) and MITM (F1 = 0.30). Gaussian Naïve Bayes

showed high recall for Backdoor and Scanning attacks but suffered from very low precision, resulting in modest F1-scores. Overall, PCA features were inadequate for preserving class separability in the ToN-IoT dataset, leading to substantial degradation in attack-wise classification performance across all models.

Table 12 presents the attack-wise performance using FastICA features on the ToN-IoT dataset. Overall, FastICA resulted in severe degradation in classification performance across all models. Random Forest and KNN achieved reasonable performance only for Normal traffic, with F1-scores of 0.79, while most attack classes exhibited near-zero F1-scores. Decision Tree showed limited effectiveness, obtaining moderate performance for Normal traffic (F1 = 0.46) and XSS (F1 = 0.09). Gaussian Naïve Bayes performed poorly, with only modest detection of Backdoor (F1 = 0.10) and DDoS (F1 = 0.27). MLP achieved moderate performance for DDoS (F1 = 0.34) but failed to classify most other attacks. Overall, FastICA features were ineffective in preserving discriminative information for the ToN-IoT dataset, leading to poor attack-wise classification performance across all classifiers.

Table 13 presents attack-wise results using Incremental PCA on the ToN-IoT dataset. Overall, all classifiers exhibited poor attack detection performance. Random Forest and KNN achieved the highest overall accuracies (65.22% and 65.18%, respectively), mainly due to correctly classifying Normal traffic (F1 = 0.79), while their performance on most attack classes remained near zero. Decision Tree showed moderate performance for Normal traffic (F1 = 0.72) but weak detection of attack categories. Gaussian NB and MLP performed poorly across almost all

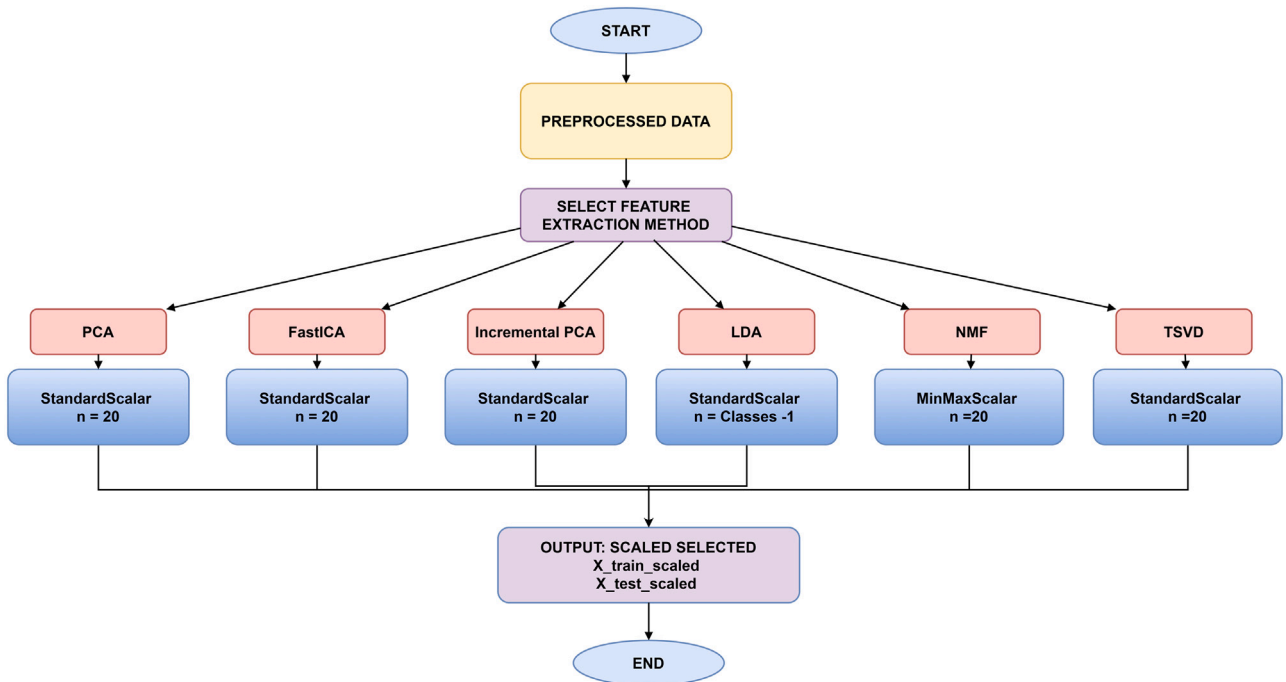


Fig. 3. Feature extraction.

Table 11

Attack-wise performance using PCA features on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	0.08	0.00	0.00	0.60	0.00	0.00	0.00	0.00	0.00	0.13	0.99	0.23	0.00	0.00	0.00
DDoS	0.21	0.01	0.02	0.57	0.01	0.01	0.75	0.12	0.21	0.52	0.24	0.33	0.12	0.03	0.05
DoS	0.26	0.08	0.12	0.37	0.90	0.53	0.35	0.90	0.50	0.00	0.00	0.00	0.38	0.92	0.53
Injection	0.01	0.01	0.01	0.03	0.02	0.02	0.01	0.02	0.01	0.00	0.00	0.00	0.07	0.01	0.02
MITM	0.20	0.58	0.30	0.14	0.50	0.22	0.01	0.46	0.02	0.03	0.49	0.06	0.06	0.57	0.11
Normal	0.82	0.65	0.73	0.89	0.68	0.77	0.95	0.35	0.51	0.99	0.11	0.20	0.65	0.49	0.56
Password	0.01	0.02	0.01	0.01	0.03	0.02	0.00	0.00	0.00	0.17	0.68	0.27	0.01	0.03	0.02
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Scanning	0.07	0.01	0.02	0.00	0.00	0.00	0.03	0.01	0.02	0.13	0.77	0.23	0.47	0.02	0.05
XSS	0.05	0.30	0.09	0.16	0.90	0.27	0.33	0.78	0.46	0.00	0.00	0.00	0.57	0.15	0.23

Table 12

Attack-wise performance comparison using FastICA on ToN-IoT Dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	0.11	0.00	0.00	0.89	0.00	0.00	0.44	0.00	0.00	0.05	1.00	0.10	0.11	0.00	0.00
DDoS	0.30	0.00	0.00	0.91	0.00	0.00	0.79	0.00	0.01	0.32	0.24	0.27	0.43	0.28	0.34
DoS	0.78	0.00	0.00	0.00	0.00	0.00	0.97	0.04	0.08	0.00	0.01	0.00	0.05	0.99	0.09
Injection	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
MITM	0.00	0.01	0.00	1.00	0.00	0.01	0.67	0.01	0.01	0.00	0.00	0.00	0.26	0.02	0.03
Normal	0.83	0.32	0.46	0.65	1.00	0.79	0.67	0.98	0.79	0.99	0.02	0.03	0.65	0.08	0.15
Password	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.01
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Scanning	0.00	0.00	0.00	0.00	0.00	0.00	0.22	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
XSS	0.05	0.77	0.09	0.17	0.00	0.00	0.13	0.15	0.14	0.00	0.00	0.00	0.90	0.07	0.13

classes, with overall accuracies of 13.43% and 12.78%, respectively. These results indicate that Incremental PCA features were ineffective for distinguishing attack classes in the ToN-IoT dataset.

Table 14 shows that LDA features perform poorly on the ToN-IoT dataset. Most attack classes, including Backdoor, Injection, MITM, Password, Ransomware, and Scanning, achieved near-zero F1-scores across all classifiers. The best performance was observed for Normal traffic, where MLP, Random Forest, and KNN achieved F1-scores of 0.80, 0.79, and 0.79, respectively. MLP also provided moderate detection for DDoS (F1 = 0.53), while Decision Tree and Gaussian NB achieved only 0.29.

Overall, LDA features are ineffective for distinguishing most attack categories in the ToN-IoT dataset.

Table 15 presents the attack-wise performance of classifiers using NMF features on the ToN-IoT dataset. Overall, all classifiers struggled to distinguish most attack categories and mainly favored the Normal class. Random Forest, KNN, and MLP achieved their highest performance for Normal traffic with F1-scores of 0.79, while Decision Tree attained an F1-score of 0.78. Gaussian NB showed poor overall performance but achieved moderate results for DDoS (F1 = 0.35) and Password attacks (F1 = 0.11). Most attack classes, including Backdoor, DoS, Injection,

Table 13

Attack-wise performance comparison using Incremental PCA on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	0.00	0.00	1.00	0.00	0.00	0.62	0.00	0.00	0.08	0.00	0.00	0.75	0.00	0.00
DDoS	0.77	0.00	0.01	0.91	0.00	0.01	0.80	0.02	0.04	0.40	0.24	0.30	0.23	0.45	0.31
DoS	0.93	0.04	0.08	0.96	0.03	0.07	0.91	0.01	0.02	0.05	0.99	0.09	0.81	0.03	0.06
Injection	0.00	0.00	0.00	0.00	0.00	0.00	0.25	0.00	0.00	0.00	0.00	0.00	0.02	0.05	0.03
MITM	0.01	0.48	0.02	0.00	0.00	0.00	0.50	0.01	0.01	0.03	0.01	0.01	0.00	0.00	0.00
Normal	0.63	0.85	0.72	0.65	1.00	0.79	0.65	1.00	0.79	0.99	0.12	0.22	0.98	0.10	0.18
Password	0.07	0.00	0.00	0.12	0.00	0.00	0.19	0.00	0.00	0.01	0.00	0.00	0.06	0.96	0.11
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Scanning	0.00	0.00	0.00	0.00	0.00	0.00	0.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
XSS	0.03	0.00	0.00	0.00	0.00	0.00	0.92	0.00	0.00	0.00	0.00	0.00	0.15	0.01	0.03

Table 14

Attack-wise performance using LDA features on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	0.50	0.00	0.00	0.75	0.00	0.00	0.31	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.00
DDoS	0.53	0.20	0.29	0.00	0.00	0.00	0.80	0.01	0.01	0.38	0.24	0.29	0.69	0.43	0.53
DoS	0.03	0.00	0.00	0.00	0.00	0.00	0.98	0.03	0.06	0.05	0.97	0.09	0.88	0.04	0.08
Injection	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
MITM	0.00	0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.01	0.00	0.00	0.00
Normal	0.76	0.24	0.37	0.65	1.00	0.79	0.66	0.99	0.79	0.98	0.12	0.22	0.66	0.99	0.80
Password	0.03	0.00	0.00	0.21	0.00	0.00	0.29	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Scanning	0.00	0.00	0.00	0.00	0.00	0.00	0.58	0.01	0.02	0.00	0.00	0.00	0.00	0.00	0.00
XSS	0.07	0.00	0.00	1.00	0.00	0.00	0.15	0.06	0.09	0.00	0.00	0.00	0.12	0.00	0.00

Table 15

Attack-wise performance using NMF features on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	0.75	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.00
DDoS	0.00	0.00	0.00	0.00	0.00	0.00	0.99	0.02	0.04	0.43	0.23	0.30	0.48	1.00	0.65
DoS	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.07	0.03	0.04
Injection	0.74	0.27	0.40	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.87	0.36	0.51
MITM	0.00	0.00	0.00	1.00	0.06	0.10	1.00	0.06	0.10	0.01	0.37	0.02	0.98	0.83	0.90
Normal	0.89	0.64	0.74	0.77	0.78	0.77	0.77	0.69	0.72	1.00	0.55	0.71	0.88	0.86	0.87
Password	0.17	0.75	0.28	0.00	0.00	0.00	0.14	0.00	0.00	0.02	0.00	0.01	0.47	0.73	0.58
Ransomware	0.01	0.01	0.01	0.67	0.00	0.00	1.00	0.00	0.00	0.09	1.00	0.17	0.46	0.96	0.62
Scanning	0.16	0.95	0.28	0.12	0.93	0.21	0.10	0.94	0.18	0.04	0.01	0.01	0.85	1.00	0.92
XSS	0.00	0.00	0.00	0.00	0.00	0.00	0.06	0.00	0.00	0.03	0.02	0.02	0.77	0.62	0.69

Ransomware, and Scanning, received near-zero F1-scores across all classifiers, indicating that NMF features were ineffective in capturing discriminative information for these attack types. Overall, NMF features resulted in limited attack detection capability, with models primarily learning the Normal traffic class.

Table 16 presents the training performance of classifiers using TSVD features on the ToN-IoT dataset. Overall, all classifiers struggled to classify most attack categories accurately. Decision Tree, Random Forest, and KNN mainly recognized Normal traffic, but achieved very low or zero F1-scores for many attacks. Gaussian NB showed moderate performance for DDoS (F1 = 0.30) and DoS (F1 = 0.09), while MLP performed relatively better for DDoS (F1 = 0.50), Password (F1 = 0.24), and XSS (F1 = 0.27). However, several attack types, including Injection, Ransomware, and Scanning, remained poorly detected by most classifiers. Overall, TSVD features were ineffective for distinguishing multiple attack categories in the ToN-IoT dataset.

Table 17 presents the training performance of classifiers using PCA features on the CIC-IoT-2022 dataset. Decision Tree, Random Forest, and KNN achieved strong performance across all attack categories, with F1-scores ranging from 0.85 to 1.00. These classifiers attained near-perfect results for HTTPFlood, Normal, and UDPFlood traffic. MLP also performed effectively, achieving perfect classification for Normal traffic (F1 = 1.00) and high F1-scores for HTTPFlood and UDPFlood (0.98–0.99), although its performance decreased for

TCPFlood (F1 = 0.82). In contrast, Gaussian NB showed comparatively weaker performance, particularly for BruteForce (F1 = 0.46), Normal (F1 = 0.56), and UDPFlood (F1 = 0.57), while achieving moderate results for HTTPFlood and TCPFlood (F1 = 0.70 and 0.68, respectively). Overall, PCA features were highly effective for Decision Tree, Random Forest, KNN, and MLP, whereas Gaussian NB remained the least effective classifier.

Table 18 presents the training performance of classifiers using FASTICA features on the CIC-IoT-2022 dataset. Decision Tree, Random Forest, and KNN achieved strong results across all attack categories, with F1-scores ranging from 0.84 to 1.00. Random Forest and KNN maintained near-perfect classification for HTTPFlood, Normal, and UDPFlood traffic, while Decision Tree also demonstrated robust performance. MLP achieved excellent results for Normal (F1 = 1.00), HTTPFlood (F1 = 0.98), and UDPFlood (F1 = 0.99), although its performance was lower for TCPFlood (F1 = 0.82). Gaussian NB showed comparatively weaker performance, particularly for Normal traffic (F1 = 0.54), but achieved moderate results for BruteForce (F1 = 0.57), HTTPFlood (F1 = 0.71), TCPFlood (F1 = 0.63), and UDPFlood (F1 = 0.76). Overall, FASTICA features were highly effective for Decision Tree, Random Forest, KNN, and MLP, whereas Gaussian NB remained the least effective classifier.

Table 19 presents the training performance of classifiers using IncrementalPCA features on the CIC-IoT-2022 dataset. Decision Tree,

Table 16

Attack-wise performance using TSVD features on ToN-IoT dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Backdoor	1.00	0.00	0.00	1.00	0.00	0.00	0.62	0.00	0.00	0.08	0.00	0.00	0.38	0.00	0.00
DDoS	0.71	0.00	0.01	0.91	0.00	0.01	0.80	0.02	0.04	0.40	0.24	0.30	0.58	0.44	0.50
DoS	0.79	0.01	0.03	0.96	0.03	0.07	0.93	0.01	0.02	0.05	0.99	0.09	0.06	0.98	0.11
Injection	0.00	0.00	0.00	0.00	0.00	0.00	0.25	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
MITM	0.00	0.00	0.00	1.00	0.01	0.01	0.50	0.01	0.01	0.03	0.01	0.01	0.13	0.10	0.11
Normal	0.82	0.72	0.77	0.65	1.00	0.79	0.65	1.00	0.79	0.99	0.12	0.22	0.92	0.11	0.20
Password	0.06	0.00	0.00	0.12	0.00	0.00	0.19	0.00	0.00	0.01	0.00	0.00	0.17	0.44	0.24
Ransomware	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Scanning	0.00	0.00	0.00	0.00	0.00	0.00	0.67	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
XSS	0.03	0.30	0.06	0.00	0.00	0.00	0.92	0.00	0.00	0.00	0.00	0.00	0.69	0.16	0.27

Table 17

Attack-wise performance using PCA Features CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.90	0.85	0.78	0.94	0.85	0.34	0.71	0.46	0.75	0.97	0.85
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.57	0.92	0.70	0.98	0.98	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.90	0.40	0.56	0.99	1.00	1.00
TCPFlood	0.88	0.81	0.85	0.87	0.82	0.85	0.91	0.80	0.85	0.68	0.68	0.68	0.99	0.70	0.82
UDPFlood	0.99	0.98	0.99	1.00	0.99	0.99	0.99	0.99	0.99	0.56	0.58	0.57	0.99	0.99	0.99

Table 18

Attack-wise Performance using FASTICA on CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.90	0.85	0.78	0.94	0.85	0.44	0.79	0.57	0.75	0.97	0.85
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.56	0.96	0.71	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.97	0.37	0.54	1.00	1.00	1.00
TCPFlood	0.88	0.81	0.84	0.87	0.83	0.85	0.91	0.80	0.85	0.59	0.69	0.63	0.99	0.70	0.82
UDPFlood	0.99	0.99	0.99	1.00	0.99	0.99	0.99	0.98	0.99	0.75	0.77	0.76	0.98	0.99	0.99

Table 19

Attack-wise performance using IncrementalPCA on CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.89	0.85	0.78	0.94	0.85	0.21	0.00	0.00	0.75	0.97	0.85
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.51	0.94	0.67	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.86	0.38	0.53	1.00	1.00	1.00
TCPFlood	0.88	0.81	0.84	0.87	0.83	0.85	0.91	0.80	0.85	0.69	0.66	0.67	0.97	0.72	0.82
UDPFlood	0.99	0.98	0.99	1.00	0.99	0.99	0.99	0.99	0.99	0.64	0.56	0.60	1.00	0.95	0.97

Random Forest, and KNN achieved excellent results, with F1-scores ranging from 0.84 to 1.00 across all attack categories. Random Forest and KNN maintained near-perfect performance for HTTPFlood, Normal, and UDPFlood traffic, while Decision Tree also showed strong classification capability. MLP performed well for most classes, achieving perfect classification for Normal traffic (F1 = 1.00) and high F1-scores for HTTPFlood (0.98) and UDPFlood (0.97), although its performance was lower for TCPFlood (0.82). In contrast, Gaussian NB exhibited the weakest performance, particularly for BruteForce traffic (F1 = 0.00), while achieving moderate results for HTTPFlood (0.67), TCPFlood (0.67), and UDPFlood (0.60). Overall, IncrementalPCA features were highly effective for tree-based models, KNN, and MLP, but less suitable for Gaussian NB.

Table 20 presents the training performance of classifiers using NMF features on the CIC-IoT dataset. Random Forest, KNN, and MLP achieved strong overall performance, particularly for HTTPFlood, Normal, and UDPFlood classes, with F1-scores reaching up to 1.00. MLP showed consistent results across most attack categories, while KNN and Random Forest performed exceptionally well for Normal traffic. Decision Tree achieved high performance for Normal and UDPFlood classes but struggled significantly with BruteForce attacks (F1 = 0.08). Gaussian NB exhibited the weakest performance overall, particularly

for BruteForce (F1 = 0.25) and TCPFlood (F1 = 0.63), although it maintained reasonable results for Normal traffic (F1 = 0.89). Overall, NMF features were most effectively utilized by Random Forest, KNN, and MLP classifiers.

Table 21 presents the training performance of classifiers using LDA features on the CIC-IoT-2022 dataset. Decision Tree, Random Forest, and KNN achieved strong performance across most attack categories, with F1-scores ranging from 0.84 to 1.00. Random Forest and KNN maintained near-perfect classification for HTTPFlood and Normal traffic. MLP also performed well, particularly for HTTPFlood and Normal classes, although its performance decreased for TCPFlood (F1 = 0.80). Gaussian NB showed comparatively weaker results, especially for BruteForce (F1 = 0.46) and UDPFlood (F1 = 0.59), but achieved reasonable performance for HTTPFlood and Normal traffic (F1 = 0.91). Overall, LDA features enabled effective learning for most classifiers, with Random Forest and KNN demonstrating the most consistent performance.

Table 22 shows the training performance of classifiers using TSVD features on the CIC-IoT-2022 dataset. Decision Tree, Random Forest, and KNN achieved excellent results, with F1-scores ranging from 0.84 to 1.00 across all classes. MLP also performed well, attaining perfect classification for Normal traffic (F1 = 1.00) and high scores for most

Table 20

Attack-wise Performance using NMF on CIC-IoT Dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.06	0.14	0.08	0.96	0.22	0.35	0.98	0.29	0.45	0.15	0.96	0.25	0.72	0.88	0.79
HTTPFlood	0.85	0.78	0.81	0.97	0.99	0.98	0.91	1.00	0.95	0.88	0.62	0.73	0.97	0.97	0.97
Normal	1.00	0.98	0.99	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.80	0.89	0.98	0.99	0.98
TCPFlood	0.93	0.71	0.81	0.61	0.84	0.71	0.96	0.75	0.84	0.58	0.68	0.63	0.99	0.67	0.80
UDPFlood	0.99	0.98	0.98	1.00	0.99	1.00	0.98	0.97	0.98	0.48	0.99	0.64	0.99	0.86	0.92

Table 21

Attack-wise performance using LDA on CIC-IoT-2022 dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.89	0.85	0.78	0.94	0.85	0.34	0.74	0.46	0.73	0.96	0.83
HTTPFlood	0.99	0.98	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.90	0.92	0.91	0.97	0.98	0.97
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.96	0.87	0.91	0.99	0.99	0.99
TCPFlood	0.87	0.81	0.84	0.87	0.82	0.84	0.91	0.79	0.85	0.79	0.67	0.72	0.95	0.69	0.80
UDPFlood	0.95	0.94	0.94	0.98	0.95	0.97	0.96	0.95	0.95	0.47	0.78	0.59	0.95	0.86	0.90

Table 22

Attack-wise performance using TSVD on CIC IoT Dataset.

Attack type	Decision tree			Random forest			KNN			Gaussian NB			MLP		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
BruteForce	0.81	0.92	0.86	0.81	0.89	0.85	0.78	0.94	0.85	0.07	0.00	0.00	0.75	0.97	0.85
HTTPFlood	0.99	0.98	0.99	0.99	0.99	0.99	0.99	0.99	0.99	0.53	0.96	0.68	0.98	0.99	0.98
Normal	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.90	0.40	0.56	1.00	1.00	1.00
TCPFlood	0.88	0.81	0.84	0.87	0.83	0.85	0.91	0.80	0.85	0.68	0.67	0.68	0.99	0.70	0.82
UDPFlood	0.99	0.99	0.99	1.00	0.99	0.99	0.99	0.99	0.99	0.48	0.57	0.52	0.99	0.99	0.99

attack types, although its performance was lower for TCPFlood (F1 = 0.82). In contrast, Gaussian NB showed comparatively weaker performance, particularly for BruteForce traffic (F1 = 0.00), while achieving moderate results for the remaining classes. Overall, TSVD features are effective for most classifiers, especially tree-based models, KNN, and MLP.

5.3. FS vs FE comparative analysis

This section provides a comprehensive comparative analysis of feature selection (FS) and feature extraction (FE) techniques evaluated across two benchmark IoT IDS datasets: ToN-IoT and CIC-IoT-2022. The analysis considers five feature selection methods (Mutual Information, ANOVA, Chi-Square, Random Forest Importance, and RFE), six feature extraction methods (PCA, FastICA, Incremental PCA, LDA, NMF, and TSVD), and five classifiers: Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN), Gaussian Naïve Bayes (GNB), and Multi-Layer Perceptron (MLP), and examines the trade-offs in classification performance, robustness, and classifier compatibility. We removed the Random Forest column from Tables 4 and 10 because the feature subsets were obtained using Random Forest Feature Importance. Including Random Forest as a classifier in these tables would favor the model used for feature selection and could bias the comparison. Therefore, only the other classifiers are reported to provide a fair evaluation.

5.3.1. Performance on the ToN-IoT dataset

On the ToN-IoT dataset, feature selection techniques consistently outperformed feature extraction methods in terms of overall attack detection reliability, although no FS method produced uniformly perfect results across all ten attack categories. Performance instead varied considerably depending on the specific FS technique and classifier pairing. Chi-Square feature selection (Table 2) produced the strongest and most consistent results among all FS methods on this dataset: DT, RF, and KNN achieved F1-scores of 1.00 for Backdoor and remained above 0.85 for the majority of attack classes, with MITM as the most difficult class (F1 = 0.63–0.73 across these three classifiers). RFE (Table

5) delivered comparably strong results for RF, with F1-scores reaching 0.96–1.00 for Backdoor, DoS, DDoS, Normal, Password, and Scanning; DT followed a similar pattern but with somewhat lower scores for DDoS (F1 = 0.89) and Password (F1 = 0.92). Injection (DT F1 = 0.63) and XSS (DT F1 = 0.34, RF F1 = 0.54) remained comparatively weak for both classifiers. Mutual Information (Table 1) yielded strong RF performance for Backdoor, DDoS, and Injection (F1 = 0.96–0.99) but a more pronounced drop for Ransomware (RF F1 = 0.58) and MITM (RF F1 = 0.72), while DT under MI struggled noticeably with Ransomware (F1 = 0.23) and MITM (F1 = 0.52). ANOVA (Table 3) was the weakest of the filter-based FS methods for tree-based models, with DT and RF both collapsing to near-zero F1-scores for Backdoor, Ransomware, and Scanning, despite reasonable performance on DDoS, Normal traffic, and XSS. For Random Forest Importance (Table 4), only DT, KNN, GNB, and MLP were evaluated; DT achieved F1-scores of 0.93–1.00 for Backdoor, DDoS, DoS, Password, and Scanning, but dropped sharply for MITM (F1 = 0.06) and XSS (F1 = 0.32).

KNN performance under feature selection was strongly method-dependent rather than uniformly reliable. Under Chi-Square and RFE, KNN reached F1-scores of 0.85–1.00 for most attack categories, with MITM again the weakest class (F1 = 0.56–0.63). Under Mutual Information and ANOVA, however, KNN performance degraded substantially for Backdoor and Scanning, with F1-scores falling to 0.01–0.14 for these two classes under both methods. GNB remained the least reliable classifier across every FS method, with most F1-scores below 0.40 regardless of technique; its best results occurred under Chi-Square for Ransomware (F1 = 0.15) and Password (F1 = 0.38). MLP showed the same method-dependence as KNN, performing best under Chi-Square (F1 = 0.83–0.96 for DDoS, Injection, Normal, and Ransomware) and worst under Mutual Information, where its Injection and Ransomware F1-scores fell to 0.01–0.03.

In contrast, feature extraction methods on the ToN-IoT dataset produced substantially degraded and inconsistent performance across all classifiers, with most models defaulting to recognizing only the Normal traffic class. PCA (Table 11) produced the least poor results among the unsupervised FE methods: RF achieved its best F1-scores

Table 23
Performance Comparison on ToN-IoT dataset.

Technique	Metric	Decision tree	Random forest	KNN	Gaussian NB	MLP
Mutual information	Accuracy	74.35	89.33	71.49	21.25	73.74
	Precision	89.54	89.52	74.42	70.52	71.88
	Recall	74.35	89.33	71.49	21.25	73.74
	F1-score	79.09	87.41	68.57	19.72	69.12
TSVD	Accuracy	48.33	65.21	65.18	13.43	16.19
	Precision	64.43	55.54	61.47	66.59	68.21
	Recall	48.33	65.21	65.18	13.43	16.19
	F1-score	50.27	51.67	51.63	16.13	17.99
ANOVA	Accuracy	76.49	79.68	66.48	11.37	71.43
	Precision	76.46	79.42	77.21	69.83	75.42
	Recall	76.49	79.68	66.48	11.37	71.43
	F1-score	75.08	74.82	63.66	13.14	69.87
Chi-Square	Accuracy	98.15	98.25	97.91	11.19	88.37
	Precision	98.19	98.30	98.07	68.80	90.74
	Recall	98.15	98.25	97.91	11.19	88.37
	F1-score	98.16	98.26	97.95	8.79	88.31
PCA	Accuracy	44.42	52.20	30.59	19.24	36.93
	Precision	56.36	65.20	68.25	68.81	49.01
	Recall	44.42	52.20	30.59	19.24	36.93
	F1-score	48.50	53.64	38.28	17.91	40.10
Incremental PCA	Accuracy	55.45	65.22	65.18	13.43	12.78
	Precision	52.93	55.32	60.68	66.59	72.81
	Recall	55.45	65.22	65.18	13.43	12.78
	F1-score	47.34	51.67	51.63	16.13	13.66
LDA	Accuracy	16.70	65.07	65.06	13.15	66.51
	Precision	54.32	50.89	56.30	65.79	50.47
	Recall	16.70	65.07	65.06	13.15	66.51
	F1-score	25.17	51.31	52.35	15.74	54.37
FastICA	Accuracy	24.08	65.08	64.34	6.48	11.19
	Precision	59.36	51.03	54.74	66.14	48.67
	Recall	24.08	65.08	64.34	6.48	11.19
	F1-score	30.34	51.33	52.70	3.76	12.05
NMF	Accuracy	63.18	65.07	65.08	10.30	65.08
	Precision	43.66	49.12	51.78	72.12	46.04
	Recall	63.18	65.07	65.08	10.30	65.08
	F1-score	50.76	51.32	51.36	10.87	51.33
RFE	Accuracy	91.25	93.11	83.77	16.56	79.30
	Precision	90.50	92.86	84.07	70.21	80.89
	Recall	91.25	93.11	83.77	16.56	79.30
	F1-score	90.44	92.22	81.13	15.78	75.66
Random forest	Accuracy	69.60	71.50	79.68	23.60	75.83
	Precision	88.52	74.45	79.42	70.23	84.43
	Recall	69.60	71.50	79.68	23.60	75.83
	F1-score	75.50	68.57	74.82	23.28	71.80

for Normal traffic (0.77), DoS (0.53), and XSS (0.27), while DT performed acceptably only for Normal traffic (F1 = 0.73) and MITM (F1 = 0.30), with most other attack categories at or near zero for every classifier. FastICA (Table 12) was the weakest FE method overall, with all five classifiers recording near-zero F1-scores for almost every attack category; only Normal traffic was detected with any reliability, by RF and KNN (F1 = 0.79). Incremental PCA (Table 13) and TSVD (Table 16) exhibited a near-identical pattern to one another: RF and KNN correctly identified Normal traffic (F1 = 0.79 for both methods) but failed almost completely on every attack class, while DT achieved moderate Normal-traffic detection (F1 = 0.72–0.77) and otherwise contributed little discriminative value; MLP under TSVD achieved its best attack-class results for DDoS (F1 = 0.50), Password (F1 = 0.24), and XSS (F1 = 0.27), but remained weak for MITM (F1 = 0.11) and Scanning (F1 = 0.00). LDA (Table 14) produced uniformly poor attack detection across all models, with almost every attack class at or near zero F1; the only consistently detected class was Normal traffic, where MLP, RF, and KNN reached F1-scores of 0.79–0.80. NMF (Table 15) likewise concentrated its detection capability on the Normal class (F1 = 0.71–0.87 across classifiers), with most attack categories, including

Backdoor, DoS, and Scanning, contributing negligible discriminative signal for the tree-based and distance-based classifiers. Overall, none of the six FE methods preserved sufficient class separability to support reliable multi-class attack detection on ToN-IoT, in marked contrast to the comparatively stronger and more consistent results obtained with Chi-Square and RFE feature selection.

In Table 23 the results on the ToN-IoT dataset show that Chi-Square feature selection consistently achieved the best performance, with Random Forest obtaining the highest accuracy of 98.25% and F1-score of 98.26%, closely followed by Decision Tree and KNN. RFE also produced strong results, whereas dimensionality reduction techniques such as PCA, TSVD, LDA, FastICA, and NMF yielded comparatively lower performance. Overall, Random Forest combined with Chi-Square feature selection provided the most effective classification performance.

5.3.2. Performance on the CIC-IoT-2022 dataset

The CIC-IoT-2022 dataset, which contains fewer but more balanced attack classes (BruteForce, HTTPFlood, Normal, TCPFlood, UDPFlood), exhibited a markedly different pattern of results. On this dataset, both

FS and FE methods yielded high performance for tree-based models and KNN, and the performance gap between the two paradigms narrowed considerably compared to ToN-IoT. Nevertheless, important distinctions persisted. Among FS methods, Mutual Information (Table 6) enabled near-perfect performance for DT, RF, and KNN ($F1 = 0.85\text{--}1.00$), and notably also for MLP, which achieved $F1$ -scores between 0.82 and 1.00, a substantial improvement over its ToN-IoT results. Random Forest Importance (Table 9), for which only DT, KNN, GNB, and MLP were evaluated, showed the same near-perfect pattern for DT and KNN ($F1 = 0.85\text{--}1.00$) and for MLP ($F1 = 0.79\text{--}1.00$). RFE (Table 10) matched this performance closely, with DT, RF, KNN, and MLP all reaching $F1$ -scores of 0.79–1.00 across every attack category. Chi-Square (Table 7) was clearly the weakest FS method on this dataset, the reverse of its position on ToN-IoT: DT and RF $F1$ -scores fell to 0.75–0.77 for BruteForce and 0.80–0.81 for TCPFlood, and MLP collapsed entirely on HTTPFlood and TCPFlood ($F1 = 0.00$), achieving an acceptable result only for Normal traffic ($F1 = 0.68$). ANOVA (Table 8) restored performance closer to the MI level for DT and RF ($F1 = 0.66\text{--}1.00$), though KNN lagged noticeably on BruteForce ($F1 = 0.29$) despite otherwise strong results ($F1$ up to 0.96 for HTTPFlood), and MLP again struggled, failing completely on TCPFlood ($F1 = 0.00$) while still reaching $F1 = 0.81$ for Normal traffic. GNB remained the weakest classifier across all FS methods on this dataset, with its best results obtained under Mutual Information ($F1$ up to 0.87 for HTTPFlood).

Among FE methods on the CIC-IoT-2022 dataset, PCA (Table 17), FastICA (Table 18), Incremental PCA (Table 19), and TSVD (Table 22) all yielded near-perfect results for DT, RF, KNN, and MLP, with the majority of $F1$ -scores in the 0.82–1.00 range; this stands in sharp contrast to the ToN-IoT results, where the same FE methods led to severe performance degradation. This difference is attributable to the simpler class structure and lower inter-class complexity of CIC-IoT-2022. LDA (Table 21) demonstrated its strongest performance on this dataset, with DT, RF, KNN, and MLP all achieving $F1$ -scores of 0.80–1.00; GNB also improved markedly under LDA, reaching its best results across all experiments ($F1 = 0.91$ for both HTTPFlood and Normal traffic), though it remained comparatively weak for BruteForce ($F1 = 0.46$) and UDPFlood ($F1 = 0.59$). NMF (Table 20) likewise proved effective on CIC-IoT-2022 for KNN and MLP across most attack types ($F1 = 0.45\text{--}1.00$, excluding BruteForce), although both DT and RF struggled noticeably with BruteForce detection ($F1 = 0.08$ and 0.35, respectively), making BruteForce the most difficult class for NMF-based feature extraction on this dataset.

The Table 24 presents a comprehensive comparison of feature selection and extraction techniques across five classifiers: Decision Tree, Random Forest, KNN, Gaussian Naive Bayes, and MLP, evaluated using Accuracy, Precision, Recall, and $F1$ -score.

Overall, most feature selection methods such as Mutual Information, TSVD, FastICA, Incremental PCA, RFE, and Random Forest-based selection consistently achieve high performance, with accuracies around 97%–98% for tree-based and KNN classifiers. Among these, RFE and Mutual Information show slightly more stable and balanced results across all evaluation metrics. In contrast, ANOVA and CHI-SQ demonstrate noticeably lower performance, particularly for Gaussian Naive Bayes and MLP, indicating weaker compatibility with these classifiers. NMF also shows mixed behavior, performing well with ensemble models but less consistently overall. Gaussian Naive Bayes is the most sensitive classifier, showing significant performance variation across feature selection methods, whereas tree-based models and KNN remain robust across nearly all transformations. Overall, the results suggest that wrapper and information-theoretic methods (such as RFE and Mutual Information) provide the most reliable feature representations for intrusion detection in this dataset.

6. Conclusion and future work

This study presented a comprehensive analysis comparing five feature selection techniques, Mutual Information, ANOVA, Chi-Square, Random Forest Importance, and Recursive Feature Elimination, and six feature extraction techniques, PCA, FastICA, Incremental PCA, LDA, NMF, and TSVD, for IoT intrusion detection systems, evaluated across the ToN-IoT and CIC-IoT-2022 benchmark datasets using five diverse classifiers.

The experimental results demonstrate that feature selection methods generally outperform feature extraction methods in terms of classification robustness, particularly on the more complex, multi-class ToN-IoT dataset, although the size of this advantage depends heavily on the specific FS technique chosen rather than holding uniformly across all configurations. Decision Tree and Random Forest achieved their strongest and most consistent detection performance under Chi-Square and RFE feature selection, but showed pronounced weaknesses for specific attack classes, such as MITM, Ransomware, and Scanning, under Mutual Information and ANOVA, indicating that classifier compatibility with a given FS method cannot be assumed to generalize across all attack categories. On the simpler CIC-IoT-2022 dataset, both Decision Tree and Random Forest remained competitive under nearly every FE method, a pattern not observed on ToN-IoT. KNN proved to be the most dataset-dependent classifier: it matched or exceeded tree-based performance on CIC-IoT-2022 under both FS and FE paradigms, but its reliability on ToN-IoT was confined largely to Chi-Square and RFE feature selection, with severe degradation under Mutual Information, ANOVA, and almost all feature extraction methods except for the Normal traffic class. GNB and MLP were the most sensitive to the choice of feature engineering method across both datasets, with $F1$ -scores for individual attack classes swinging from near-zero to above 0.85 depending solely on the FS or FE technique applied.

Among FS techniques, no single method was uniformly best across both datasets: Chi-Square delivered the strongest and most stable results on ToN-IoT but was the weakest FS method on CIC-IoT-2022, while Mutual Information, Random Forest Importance, and RFE provided consistently solid, though rarely top-ranked, performance on both datasets. This dataset-dependent reversal suggests that the suitability of a given filter-based FS method is closely tied to the underlying class structure and feature distributions of the target IoT network, and that FS technique selection should be validated empirically on the deployment dataset rather than assumed from prior results. Among FE techniques, LDA provided the most discriminative features when the number of classes was small, achieving its best results of the entire study on CIC-IoT-2022, but its effectiveness collapsed on the ten-class ToN-IoT dataset, confirming that the k -1 component constraint limits its scalability to complex multi-class IoT scenarios. The unsupervised FE methods, PCA, FastICA, Incremental PCA, and TSVD, performed adequately, and in several cases near-perfectly, on the simpler CIC-IoT-2022 dataset, but uniformly failed to preserve class-discriminative information on the more complex ToN-IoT dataset, where nearly all classifiers defaulted to recognizing only the Normal traffic class.

Despite the contributions of this study, several limitations point to important directions for future research. First, the study evaluated single-method feature engineering pipelines; hybrid approaches that combine feature selection with feature extraction (e.g., applying PCA to RF-selected features) may generate further performance gains, particularly for the attack classes that proved difficult for any single technique, such as MITM and Ransomware on ToN-IoT, and warrant systematic investigation. Second, deep learning-based feature engineering approaches, including autoencoders, variational autoencoders, and graph neural network-based representations, represent promising directions that were not covered in this study and may help recover the discriminative information lost by the unsupervised FE methods on complex multi-class datasets. Future work also includes the explainability and interpretability analysis of the selected and extracted features,

Table 24
Performance Comparison on CIC-IoT-2022 Dataset.

Technique	Metric	Decision tree	Random forest	KNN	Gaussian NB	MLP
Mutual information	Accuracy	98.14	98.12	98.17	82.78	97.74
	Precision	98.14	98.15	98.28	85.91	98.00
	Recall	98.14	98.14	98.17	82.78	97.74
	F1-score	98.15	98.14	98.18	83.24	97.70
TSVD	Accuracy	98.08	98.11	98.13	62.11	97.76
	Precision	98.12	98.14	98.24	70.98	98.02
	Recall	98.08	98.11	98.13	62.11	97.76
	F1-score	98.08	98.11	98.14	59.20	97.70
ANOVA	Accuracy	94.26	94.28	91.25	51.64	73.34
	Precision	95.12	95.15	91.11	64.32	77.40
	Recall	94.26	94.28	91.25	51.64	73.34
	F1-score	94.44	94.46	90.78	45.24	69.75
CHI-SQ	Accuracy	96.06	96.12	95.54	28.02	51.09
	Precision	96.24	96.21	95.86	81.19	29.67
	Recall	96.06	96.12	95.54	28.02	51.09
	F1-score	96.11	96.14	95.61	37.28	34.82
FastICA	Accuracy	98.07	98.12	98.13	63.64	97.76
	Precision	98.12	98.15	98.24	77.01	98.02
	Recall	98.07	98.12	98.13	63.64	97.76
	F1-score	98.07	98.12	98.13	61.17	97.70
Incremental PCA	Accuracy	98.06	98.11	98.14	60.33	97.78
	Precision	98.11	98.14	98.24	69.29	98.00
	Recall	98.06	98.11	98.14	60.33	97.78
	F1-score	98.07	98.12	98.14	57.21	97.74
NMF	Accuracy	85.84	95.99	95.92	73.09	96.43
	Precision	90.18	96.64	96.15	89.40	96.67
	Recall	85.84	95.99	95.92	73.09	96.43
	F1-score	87.82	95.38	95.22	78.68	96.35
Random Forest FS	Accuracy	98.07	98.14	98.14	63.35	97.62
	Precision	98.12	98.24	98.24	73.69	97.84
	Recall	98.07	98.14	98.14	63.35	97.62
	F1-score	98.07	98.14	98.14	61.78	97.57
RFE	Accuracy	98.14	98.12	98.17	82.78	97.55
	Precision	98.19	98.15	98.28	85.91	97.57
	Recall	98.14	98.12	98.17	82.78	97.55
	F1-score	98.14	98.12	98.18	83.24	97.50
LDA	Accuracy	97.91	98.01	98.03	87.54	96.63
	Precision	97.95	98.04	98.13	90.32	96.86
	Recall	97.91	98.01	98.03	87.54	96.63
	F1-score	97.91	98.01	98.03	88.50	96.58
PCA	Accuracy	98.07	98.11	98.14	63.35	97.61
	Precision	98.12	98.15	98.24	73.69	97.86
	Recall	98.07	98.11	98.14	63.35	97.61
	F1-score	98.07	98.12	98.14	61.78	97.55

using tools such as SHAP values, which would provide deeper insights into the decision-making process of IoT IDS models and help explain the strong dataset-dependence observed between Chi-Square and the other FS methods, enhancing their trustworthiness for deployment in critical infrastructure.

CRediT authorship contribution statement

Monika Vishwakarma: Writing – original draft, Validation, Formal analysis. **Divyansh Dwivedi:** Software, Methodology, Investigation, Data curation. **Nishtha Kesswani:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We used benchmarks datasets which are freely accessible.

References

- [1] Mishra R, Mishra A. Current research on internet of things (IoT) security protocols: A survey. *Comput Secur* 2025;104310.
- [2] Wei C, Xie G, Diao Z. A lightweight deep learning framework for botnet detecting at the IoT edge. *Comput Secur* 2023;129:103195.
- [3] Yalli JS, Hasan MH, Jung LT, Yerima AI, Aliyu DA, Maiwada UD, Al-Selwi SM, Shaikh MU. A systematic review for evaluating IoT security: A focus on authentication, protocols and enabling technologies. *IEEE Internet Things J* 2025.
- [4] Thakkar A, Lohiya R. A review on machine learning and deep learning perspectives of IDS for IoT: Recent updates, security issues, and challenges. *Arch Comput Methods Eng* 2021;28(4).
- [5] Ghubaihs A, Yang Z, Erbad A, Jain R. LEMDA: A novel feature engineering method for intrusion detection in IoT systems. *IEEE Internet Things J* 2023;11(8):13247–56.
- [6] Pham VT, Huu TV, Nguyen MT, Le H-C. Advanced feature processing for IoT-based intrusion detection system. In: 2023 RIVF international conference on computing and communication technologies. 2023, p. 37–42. <http://dx.doi.org/10.1109/RIVF60135.2023.10471837>.
- [7] Janaki K, Ravi Kumar DV, Murali Krishna CV, Sravani A, Moses G. Enhancing intrusion detection with advanced feature extraction through machine learning and deep learning methods. In: 2024 international conference on intelligent systems for cybersecurity. 2024, p. 1–6. <http://dx.doi.org/10.1109/ISCS61804.2024.10581050>.
- [8] Albulayhi K, Abu Al-Haija Q, Alsuhibany SA, Jillepalli AA, Ashrafuzzaman M, Sheldon FT. IoT intrusion detection using machine learning with a novel high performing feature selection method. *Appl Sci* 2022;12(10):5015.

- [9] Vuong T-C, Tran H, Trang MX, Ngo V-D, Luong TV. A comparison of feature selection and feature extraction in network intrusion detection systems. In: 2022 Asia-Pacific signal and information processing association annual summit and conference. 2022, p. 1798–804. <http://dx.doi.org/10.23919/APSIPAASC55919.2022.9979923>.
- [10] Li J, Othman MS, Chen H, Yusuf LM. Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning. *J Big Data* 2024;11(1):36.
- [11] Pathak M, Patel A, Shah B. IoT intrusion detection: Evaluating ML-based IDS on image and network traffic datasets. In: 2023 IEEE international symposium on smart electronic systems. 2023, p. 57–62. <http://dx.doi.org/10.1109/iSES58672.2023.00023>.
- [12] Natarajan B, Bose S, Maheswaran N, Logeswari G, Anitha T. A new high-performance feature selection method for machine learning-based IOT intrusion detection. In: 2023 12th international conference on advanced computing (iCoAC). 2023, p. 1–8. <http://dx.doi.org/10.1109/iCoAC59537.2023.10249916>.
- [13] Dahou A, Abd Elaziz M, Chelloug SA, Awadallah MA, Al-Betar MA, Al-Qaness MA, Forestiero A. Intrusion detection system for IoT based on deep learning and modified reptile search algorithm. *Comput Intell Neurosci* 2022;2022(1):6473507.
- [14] Walling S, Lodh S. Network intrusion detection system for IoT security using machine learning and statistical based hybrid feature selection. *Secur Priv* 2024;7(6):e429.
- [15] Fatani A, Abd Elaziz M, Dahou A, Al-Qaness MA, Lu S. IoT intrusion detection system using deep learning and enhanced transient search optimization. *IEEe Access* 2021;9:123448–64.
- [16] Moustafa N, Slay J. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 military communications and information systems conference. 2015, p. 1–6. <http://dx.doi.org/10.1109/MilCIS.2015.7348942>.
- [17] Moustafa N, Slay J. The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inf Secur J: A Glob Perspect* 2016;25(1–3):18–31.
- [18] Shone N, Ngoc TN, Phai VD, Shi Q. A deep learning approach to network intrusion detection. *IEEE Trans Emerg Top Comput Intell* 2018;2(1):41–50.
- [19] Ngo V-D, Vuong T-C, Van Luong T, Tran H. Machine learning-based intrusion detection: feature selection versus feature extraction. *Clust Comput* 2024;27(3):2365–79.
- [20] Sarhan M, Layeghy S, Moustafa N, Gallagher M, Portmann M. Feature extraction for machine learning-based intrusion detection in IoT networks. *Digit Commun Netw* 2024;10(1):205–16.
- [21] Vo H-N-Y, Nguyen D-T, Nguyen X-H, Le K-H. IoT intrusion detection: A comprehensive benchmark of feature selection and machine learning models. In: Conference on information technology and its applications. Springer; 2025, p. 709–20.
- [22] Alsaedi A, Moustafa N, Tari Z, Mahmood A, Anwar A. TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems. *IEEE Access* 2020;8:165130–50.
- [23] Moustafa N. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustain Cities Soc* 2021;72:102994.
- [24] Booi TM, Chiscop I, Meeuwissen E, Moustafa N, Den Hartog FT. ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets. *IEEE Internet Things J* 2021;9(1):485–96.
- [25] Almotairi A, Atawneh S, Khashan OA, Khafajah NM. Enhancing intrusion detection in IoT networks using machine learning-based feature selection and ensemble models. *Syst Sci Control Eng* 2024;12(1):2321381.
- [26] Zhang D, Huang D, Chen Y, Lin S, Li C. A lightweight IoT intrusion detection method based on two-stage feature selection and Bayesian optimization. *AIMS Electron Electr Eng* 2025;9(3).
- [27] Dadkhah S, Mahdikhani H, Danso PK, Zohourian A, Truong KA, Ghorbani AA. Towards the development of a realistic multidimensional IoT profiling dataset. In: 2022 19th annual international conference on privacy, security & trust. IEEE; 2022, p. 1–11.
- [28] Kostas K, Just M, Lones MA. Externally validating the IoTDevID device identification methodology using the CIC IoT 2022 dataset. 2023, arXiv preprint [arXiv:2307.08679](https://arxiv.org/abs/2307.08679).
- [29] Neto ECP, Dadkhah S, Ferreira R, Zohourian A, Lu R, Ghorbani AA. CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors* 2023;23(13):5941.
- [30] Kim TK. Understanding one-way ANOVA using conceptual figures. *Korean J Anesthesiol* 2017;70(1):22.
- [31] f.classif — scikit-learn 1.7.2 documentation. 2025, URL https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.f_classif.html#, [Online; Accessed 06 November 2025].
- [32] Vergara JR, Estévez PA. A review of feature selection methods based on mutual information. *Neural Comput Appl* 2014;24(1):175–86.
- [33] Peng H, Long F, Ding C. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Trans Pattern Anal Mach Intell* 2005;27(8):1226–38.
- [34] Franke TM, Ho T, Christie CA. The chi-square test: Often used and more often misinterpreted. *Am J Eval* 2012;33(3):448–58.
- [35] chi2 — scikit-learn 1.7.2 documentation. 2025, URL https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.chi2.html, [Online; Accessed 06 November 2025].
- [36] Breiman L. Random forests. *Mach Learn* 2001;45(1):5–32.
- [37] Louppe G. Understanding random forests: From theory to practice. Université de Liège (Belgium); 2014.
- [38] Guyon I, Weston J, Barnhill S, Vapnik V. Gene selection for cancer classification using support vector machines. *Mach Learn* 2002;46(1):389–422.
- [39] Abdi H, Williams LJ. Principal component analysis. *Wiley Interdiscip Rev: Comput Stat* 2010;2(4):433–59.
- [40] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, et al. Scikit-learn: Machine learning in python. *J Mach Learn Res* 2011;12:2825–30.
- [41] Hyvärinen A, Oja E. Independent component analysis: algorithms and applications. *Neural Netw* 2000;13(4–5):411–30.
- [42] Balakrishnama S, Ganapathiraju A. Linear discriminant analysis—a brief tutorial. *Inst Signal Inf Process* 1998;18(1998):1–8.
- [43] Cichocki A, Phan A-H. Fast local algorithms for large scale nonnegative matrix and tensor factorizations. *IEICE Trans Fundam Electron Commun Comput Sci* 2009;92(3):708–21.
- [44] Halko N, Martinsson P-G, Tropp JA. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Rev* 2011;53(2):217–88.